INF1037
Programação em C

2026.1 - Turma 33C
Prof. Adriano Branco
Aula 21: 18/05/2026

Slides da aula da Profa. Paula Rodrigues

Pontifícia Universidade Católica do Rio de Janeiro
Departamento de Informática

Conteúdo Programático

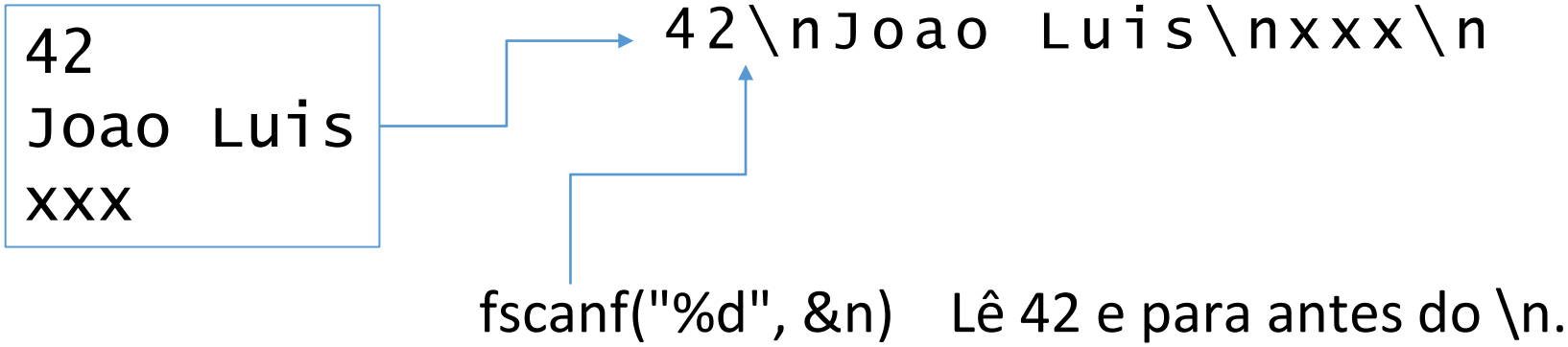
- Tipos Estruturados 
- Vetor de Tipos Estruturados 
- União e Enumeração 
- Arquivo Binário
- Lista
- Recursão
- Segunda Prova (P2)

Voltando em arquivos de texto!

Na leitura, o que fica no buffer?

Após qualquer leitura, o **\n** do final da linha fica no buffer esperando o próximo formato.

Arquivo:



Quem lida com o \n residual?

Formato	Pula whitespace à esquerda?
%d, %s, %f	Sim - não tem problema
%c, %[^\n], fgets(), getc()	Não - lê o **\n** como dado

Solução:

```
fscanf(arq, "  %[^\n]", nome);
fscanf(arq, "  %c", &c);
```

Observação: **%[^\n]** também deixa o **\n** no buffer, a mesma regra vale para o formato seguinte.

DEPARTAMENTO DE INFORMÁTICA
PUC·RIO

Arquivo Binário

Pontifícia Universidade Católica do Rio de Janeiro
Departamento de Informática

Relembrando: Arquivos na Linguagem C

- A linguagem C oferece os serviços de leitura e escrita de informações em arquivos através de um conjunto de funções.
- Principais serviços:
  - **Abertura de arquivo:** o sistema operacional encontra o arquivo com o nome dado e prepara o *buffer* na memória.
  - **Leitura do arquivo:** o sistema operacional recupera o trecho solicitado do arquivo. Como o *buffer* contém parte da informação do arquivo, parte ou toda a informação solicitada pode vir do *buffer*.
  - **Escrita no arquivo:** o sistema operacional acrescenta ou altera o conteúdo do arquivo. A alteração é feita inicialmente no *buffer* para depois ser transferida para o disco.
  - **Fechamento de arquivo:** toda a informação constante do *buffer* é atualizada no disco (ou descartada, se for um arquivo de leitura) e a área do *buffer* utilizada na memória é liberada.

Relembrando: Arquivos na Linguagem C

- Uma das informações que é mantida pelo sistema operacional é um ***cursor*** que indica a posição de trabalho no arquivo:
  - O arquivo representa uma memória sequencial.
  - O *cursor* indica a posição corrente nesta memória.

Relembrando: Conjunto de função em C para manipulação de arquivos

- Todas as funções da biblioteca padrão de C que manipulam arquivos encontram-se na **`<stdio.h>`**.

- **`FILE`** é um tipo definido pela biblioteca padrão que representa uma abstração do arquivo.

Relembrando: Conjunto de função em C para manipulação de arquivos

- A função básica para abrir um arquivo é fopen:

Devemos passar o nome do arquivo a ser aberto. O nome do arquivo pode ser relativo, e o sistema procura o arquivo a partir do diretório atual, ou pode ser absoluto, e, para tanto, especificamos o nome completo do arquivo, incluindo os diretórios, desde o diretório raiz.

```
FILE * fopen (char* nome_arquivo, char* modo);
```

Tipo de retorno: ponteiro para o tipo `FILE`, e todas as operações subsequentes nesse arquivo receberão o ponteiro como parâmetro de entrada.
Se o arquivo não puder ser aberto, a função tem como retorno o valor `NULL`.

Existem diferentes modos de abertura de um arquivo. Podemos abrir para *leitura* ou para *escrita*, e devemos especificar se o arquivo está em modo *texto* ou em modo *binário*.
O parâmetro modo da função fopen é uma cadeia de caracteres em que se espera a ocorrência de caracteres que identifiquem o modo de abertura. Os caracteres Interpretados no modo são:

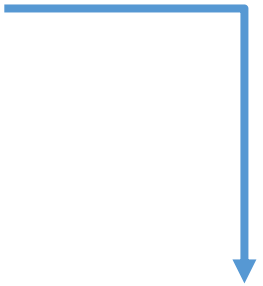
r (*read*) – modo para leitura
w (*write*) – modo para escrita
a (*append*) – modo para escrita ao final do existente

t (*text*) – modo texto
b (*binary*) – modo binário

Se quisermos abrir um arquivo para poder simultaneamente ler e escrever, acrescentamos O caractere + no modo de abertura. Assim, `r+` indica leitura e escrita em um arquivo já existente e `w+` indica leitura e escruta em um novo arquivo,

Relembrando: Modos de abertura de arquivo

Modo	Descrição
r	Abre um arquivo existente para leitura.
w	Cria um arquivo para gravação. Se o arquivo já existe, descarta o conteúdo atual.
a	Acréscimo; abre ou cria um arquivo para gravação no final do arquivo.
r+	Abre um arquivo existente para atualização (leitura e gravação).
w+	Cria um arquivo para atualização. Se o arquivo já existe, descarta o conteúdo atual.
a+	Acréscimo: abre ou cria um arquivo para atualização; a gravação é feita no final do arquivo.
rb	Abre um arquivo existente para leitura no modo binário.
wb	Cria um arquivo para gravação no modo binário. Se o arquivo já existe, descarta o conteúdo atual.
ab	Acréscimo; abre ou cria um arquivo para gravação no final do arquivo no modo binário.
rb+	Abre um arquivo existente para atualização (leitura e gravação) no modo binário.
wb+	Cria um arquivo para atualização no modo binário. Se o arquivo já existir, descarta o conteúdo atual.
ab+	Acréscimo: abre ou cria um arquivo para atualização no modo binário; a gravação é feita no final do arquivo.



- Modos de abertura para **binário**:
  - "`rb`": Leitura binária
  - "`wb`": Escrita binária
  - "`ab`": Adição binária (escreve no final)
  - "`r+b`" ou "`rb+`": Leitura/Escrita (não apagam permite modificar)
  - "`w+b`" ou "`wb+`": Leitura/Escrita (cria/sobrescreve)

Relembrando: Conjunto de função em C para manipulação de arquivos

- Quando abrir um arquivo, testamos o sucesso da abertura antes de qualquer outra operação:

Abrindo para leitura no modo binário.

```
...
FILE* fp = fopen("entrada.bin", "rb");
if (fp == NULL) {
  printf("Erro na abertura do arquivo.\n");
  exit(1);
}
...
```

Lembrete fundamental:
Sempre verificar se **`fopen`** retornou **`NULL`**!

Relembrando: Conjunto de função em C para manipulação de arquivos

- Quando abrir um arquivo, testamos o sucesso da abertura antes de qualquer outra operação:

Abrindo para escrita no modo binário.

```
...
FILE* fp = fopen("saida.bin", "wb");
if (fp == NULL) {
  printf("Erro na abertura do arquivo.\n");
  exit(1);
}
...
```

Lembrete fundamental: Se o arquivo já existe, ele é sobrescrito (cuidado!).

Relembrando: Conjunto de função em C para manipulação de arquivos

- Após ler/escrever as informações de um arquivo, devemos fechá-lo. Para fechar um arquivo, devemos usar a função `fclose`, que espera como parâmetro o ponteiro do arquivo que se deseja fechar.
- O protótipo da função é:



Para cada **`fopen`**, deve haver um **`fclose`**.

Garante que os buffers de saída sejam descarregados (*flushed*) para o disco.

```
int fclose (FILE* fp);
```

Tipo de retorno: 0 (zero) se o arquivo for fechado com sucesso, ou a constante `EOF` (definida pela biblioteca) se houver a ocorrência de um erro.

Arquivos em modo binário

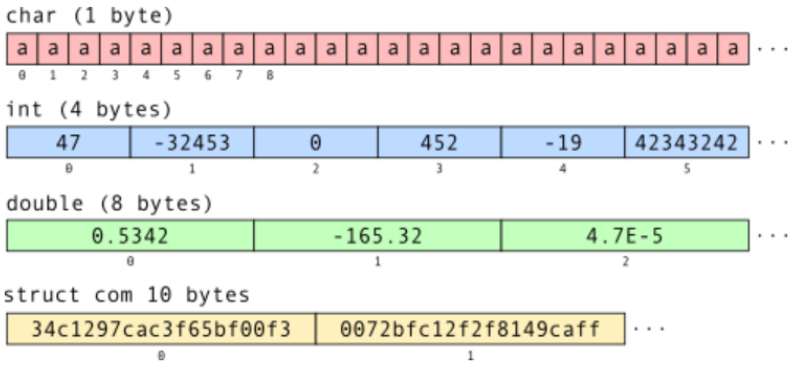
- Os arquivos em modo binário servem para salvar (e depois recuperar) o conteúdo da memória principal diretamente no disco.

- A memória é salva copiando-se o conteúdo de cada `byte` da memória para o arquivo.

- **Verificação de Erros e I/O:**
  - `ferror(FILE *stream)`: Retorna true se um erros de leitura/escrita ocorreu.
  - `feof(FILE *stream)`: Retorna true se o final do arquivo (EOF) foi atingido.

Por que Arquivos Binários?

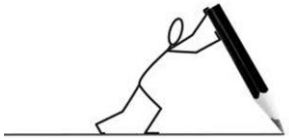
- **Eficiência:** Armazenamento mais compacto (sem conversão para caracteres).
- **Velocidade:** Leitura/Escrita mais rápida.
- **Precisão:** Preservação exata do tipo de dado (por exemplo, um `float` de 4 bytes é lido como 4 bytes, e não como uma string formatada).
- **Uso prático:** Salvamento de imagens, modelos 3D etc.



Arquivos em modo binário

- As funções `fwrite()` e `fread()` são as funções mais importantes para arquivos binários. Elas movem blocos de bytes diretamente entre a memória RAM e o arquivo.

`fwrite()` e `fread()`





Arquivos em modo binário: Escrita

- Para escrever (salvar) dados em arquivos binários, usa-se a função `fwrite`.

- O protótipo dessa função é:

```
int fwrite(const void * p, int tam, int nelem, FILE * fp);
```

Arquivos em modo binário: Escrita

- Para escrever (salvar) dados em arquivos binários, usa-se a função `fwrite`.

- O protótipo dessa função é:

```
int fwrite(const void * p, int tam, int nelem, FILE * fp);
```

Ponteiro para o bloco de dados na memória, ou seja,
endereço de memória cujo conteúdo deseja-se salvar em arquivo.

Arquivos em modo binário: Escrita

- Para escrever (salvar) dados em arquivos binários, usa-se a função `fwrite`.

- O protótipo dessa função é:

```
int fwrite(const void * p, int tam, int nelem, FILE * fp);
```

O tamanho (em bytes) de cada elemento a ser escrito (use `sizeof(tipo)`)

Arquivos em modo binário: Escrita

- Para escrever (salvar) dados em arquivos binários, usa-se a função `fwrite`.

- O protótipo dessa função é:

```
int fwrite(const void * p, int tam, int nelem, FILE * fp);
```

O número de elementos a serem escritos

Arquivos em modo binário: Escrita

- Para escrever (salvar) dados em arquivos binários, usa-se a função `fwrite`.

- O protótipo dessa função é:

```
int fwrite(const void * p, int tam, int nelem, FILE * fp);
```

Ponteiro para o arquivo binário para o qual o conteúdo será copiado (na posição que segue o cursor).
Na prática, o ponteiro `FILE*` retornado por `fopen()`.

Arquivos em modo binário: Escrita

- Para escrever (salvar) dados em arquivos binários, usa-se a função `fwrite`.

- O protótipo dessa função é:

```
int fwrite(const void * p, int tam, int nelem, FILE * fp);
```

O número de itens **escritos com sucesso**.

Arquivos em modo binário: Leitura

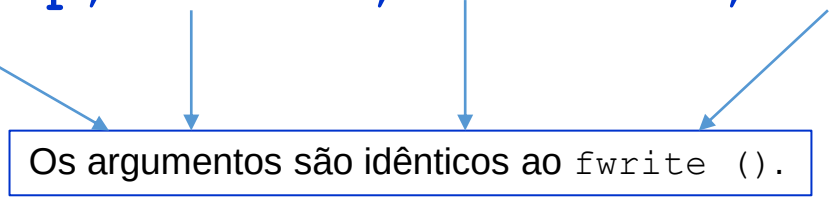
- Para ler (recuperar) dados em arquivos binários, usa-se a função `fread`.
- O conteúdo do disco é copiado para o endereço de memória que é passado como parâmetro.
- O protótipo dessa função é:

```
int fread(void * p, int tam, int nelem, FILE * fp);
```

Arquivos em modo binário: Leitura

- Para ler (recuperar) dados em arquivos binários, usa-se a função `fread`.
- O conteúdo do disco é copiado para o endereço de memória que é passado como parâmetro.
- O protótipo dessa função é:

```
int fread(void * p, int tam, int nelem, FILE * fp);
```



Arquivos em modo binário: Leitura

- Para ler (recuperar) dados em arquivos binários, usa-se a função `fread`.
- O conteúdo do disco é copiado para o endereço de memória que é passado como parâmetro.
- O protótipo dessa função é:

```
int fread(void * p, int tam, int nelem, FILE * fp);
```

O número de itens **lidos com sucesso**.

Exemplo

- Exemplo 01: Salvar em um vetor de 10 inteiros diretamente

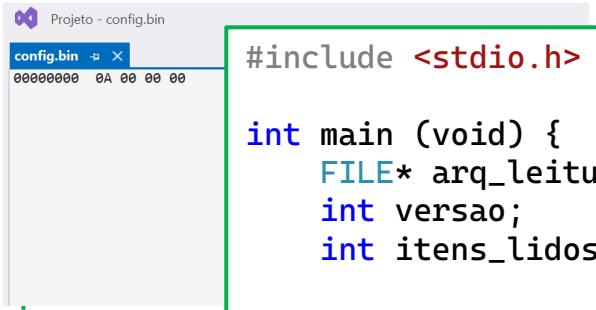
```c
#include <stdio.h>

int main(void) {
    FILE* f_ent_bin = fopen("arq_saida.bin", "wb");
    if (f_ent_bin == NULL) {
        printf("Nao foi possivel abrir o arquivo para escrita.\n");
        exit(1);
    }
    int dados[10] = {10,20,30,40,50,60,70,80,90,100};
    fwrite(dados, sizeof(int), 10, f_ent_bin);
    fclose(f_ent_bin);
    return 0;
}
```



Exemplo

- Exemplo 02: Ler a versão de um software que está armazenado em um arquivo como int.




```c
#include <stdio.h>

int main (void) {
    FILE* arq_leitura = fopen("config.bin", "rb");
    int versao;
    int itens_lidos;

    if (arq_leitura == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }
    /* Tenta ler 1 item (int), de tamanho sizeof(int), para o endereço de 'versao' */
    itens_lidos = fread(&versao, sizeof(int), 1, arq_leitura);

    if (itens_lidos == 1) {
        printf("Leitura bem-sucedida! Versão do Software: %d\n", versao);
    }
    else {
        printf("Erro de leitura ou arquivo vazio.\n");
    }

    fclose(arq_leitura);
    return 0;
}
```

Exemplo

- Exemplo 03: O verdadeiro poder do binário é salvar um registro inteiro em uma única chamada de função.

```c
#include <stdio.h>

typedef struct aluno Aluno;
struct aluno {
    int matricula;
    char nome[50];
    float media;
};

int main (void) {
    FILE* f_ent_bin = fopen("arq_saida.bin", "wb");
    Aluno a1 = { 12345, "Carlos", 8.5 };
    fwrite(&a1, sizeof(Aluno), 1, f_ent_bin); /* Salva a estrutura completa */

    fclose(f_ent_bin);
    return 0;
}
```

Como é uma ***struct***, tem que passar com o ***&***.

Exemplo

- Exemplo 04: Ler um vetor de float de tamanho 100

```c
#include <stdio.h>

#define NUM_AMOSTRAS 100

int main(void) {
    FILE* arq_leitura = fopen("dados.bin", "rb");
    float amostras[NUM_AMOSTRAS];
    int itens_lidos;

    if (arq_leitura == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        exit(1);
    }

    ...
}
```

Exemplo

- Exemplo 04: Ler um vetor de float de tamanho 100

```
...
    /* Tenta ler NUM_AMOSTRAS itens(floats), de tamanho sizeof(float),
    para o endereço do primeiro elemento do vetor 'amostras'.*/
    itens_lidos = fread(amostras, sizeof(float), NUM_AMOSTRAS, arq_leitura);

    if (itens_lidos == NUM_AMOSTRAS) {
        printf("Todos os 100 dados lidos com sucesso.\n");
        printf("Primeira amostra: %.2f\n", amostras[0]);
        printf("Última amostra: %.2f\n", amostras[NUM_AMOSTRAS - 1]);
    }
    else {
        printf("AVISO: Esperava-se ler %d itens, mas apenas %d foram lidos.\n",
            NUM_AMOSTRAS, itens_lidos);
        /* O vetor 'amostras' agora contém os 'itens_lidos' que existiam.*/
    }

    fclose(arq_leitura);
    return 0;
}
```

Como é um ***vetor***, tem que passar sem o ***&***.

Exemplo

- Exemplo 05: Lendo estruturas sequencialmente

```c
#include <stdio.h>
#include <string.h>

typedef struct aluno Aluno;
struct aluno {
    int matricula;
    char nome[50];
    float media;
};

int main(void) {
    FILE* arq_entrada = fopen("alunos.db", "rb");
    Aluno aluno_lido;
    int contador = 0;
    int itens_lidos;

    if (arq_entrada == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }

    ...
```

Exemplo

- Exemplo 05: Lendo estruturas sequencialmente

```
...

    printf("--- Registros de Alunos ---\n");
    /* Loop que lê um registro de Aluno por vez, até que fread retorne 0 */

    while (1) {
        /* Tenta ler 1 item(Aluno), de tamanho sizeof(Aluno), para &aluno_lido*/
        itens_lidos = fread(&aluno_lido, sizeof(Aluno), 1, arq_entrada);

        if (itens_lidos == 1) {
            /* Leitura bem - sucedida */
            contador++;
            printf("Registro %d: Matricula: %d, Nome: %s, Media: %.1f\n",
                contador, aluno_lido.matricula, aluno_lido.nome, aluno_lido.media);
        }
         ...
```

Exemplo

- Exemplo 05: Lendo estruturas sequencialmente

```
...

        else if (feof(arq_entrada)) {
            /* Fim do arquivo(EOF) foi atingido. */
            printf("Fim dos dados. Registros lidos: %d\n", contador);
            break;
        }
        else {
            /* Algum outro erro de leitura(não é EOF).*/
            printf("Erro inesperado durante a leitura.\n");
            break;
        }
    }

    fclose(arq_entrada);
    return 0;
}
```

Arquivos em modo binário: Acesso aleatório

- Relembrando, em arquivos textuais, a leitura é feita de forma sequencial, lendo da primeira posição para a última.
- Em arquivos binários:
  - É possível recuperar apenas parte da informação armazenada;
  - O programador tem o controle de quantos bytes ocupa cada informação armazenada no arquivo; e
  - É possível alterar a posição do cursor do arquivo e colocá-lo na posição desejada para ler uma informação. Ou seja, podemos ir diretamente para qualquer ponto.
- A função que nos permite movimentar o cursor do arquivo tem o seguinte protótipo:

```
int fseek (FILE* fp , long offset , int origem );
```

•SEEK_CUR: Posição atual.
•SEEK_END: Final do arquivo.

Arquivos em modo binário: Acesso aleatório

- A função que nos permite movimentar o cursor do arquivo tem o seguinte protótipo:

```
int fseek (FILE* fp , long offset , int origem);
```

Arquivo no qual estamos posicionando o cursor.

Número de bytes a mover.

Indica em relação a que posição estamos avançando o cursor :
- `SEEK_SET`: move a partir do início do arquivo → usar valores positivos.
- `SEEK_CUR`: move a partir da posição atual → valores positivos avançam e negativos retrocedem.
- `SEEK_END`: move a partir do final do arquivo → usar valores negativos para retroceder.

Arquivos em modo binário: Acesso aleatório

- Exemplo de uso do `fseek` – pensando no Exemplo 05:

```
int N = 5; /* quero o 5º Aluno (índice 4)*/
fseek(arquivo, N * sizeof(Aluno), SEEK_SET);
fread(&aluno_n, sizeof(Aluno), 1, arquivo);
```

Prática

1. Implemente um programa para criar um arquivo binário para armazenar o cadastro de 5 alunos. Cada registro no arquivo deve ter:

- Nome representado por uma cadeia de caracteres de 80 caracteres (ocupando 81 bytes no arquivo para armazenar também o '\0')
- Notas representadas por 3 valores do tipo float.

Você pode assumir a extensão .dat para seu arquivo binário de saída.

2. Implemente um outro programa para capturar um valor inteiro fornecido pelo usuário (de 0 a 4), representando o registro. Lendo o arquivo binário criado na questão anterior, exiba o nome do aluno e sua média, correspondente ao número do registro.